

Proyecto del 2.º curso de AOC2: Jerarquía de la memoria de datos

Equipo AOC2, EINA, Univ. zaragoza

Fecha límite de envío: 12 de mayo a las 9:00

LÉAME

- Ahorre tiempo leyendo atentamente esta Guía.

– Comprenderá cuestiones clave que posiblemente aún no le resulten claras a pesar de la teoría y los problemas planteados en clase. – Le ayudará a organizar y redactar su informe.

- No cambie el nombre de los componentes y señales utilizados en la fuente VHDL proporcionada código sin consultarlo con nosotros.

Todos los materiales (archivos fuente, diapositivas, problemas, guías...) están sujetos a los derechos de propiedad intelectual de la Universidad de Zaragoza. La violación de estos derechos puede conllevar consecuencias legales.

- Problemas de idioma: AOC2 ha sido declarado un curso ELF (English Language Friendly). Los principales libros de referencia y la mayoría de los materiales escritos internos se imparten en inglés. Aún estamos en transición desde un curso no ELF, por lo que encontrará símbolos y comentarios en español en todo el código fuente VHDL proporcionado. No dude en consultar con el profesorado si su lengua materna no es el español y tiene algún problema con esto.

- Informa cualquier error tipográfico o equivocación que encuentres y pregunta sobre cualquier cosa que necesites, pero ten en cuenta que estaremos encantados de ayudarte. deseché las preguntas que ya están respondidas en esta guía.

- ☐ Las casillas de verificación a lo largo de esta Guía constituyen la lista de verificación que debe completar antes de enviar

Poniendo en marcha tu proyecto.

Contenido

1 Resumen	2
2 Sistema de base 2.1	2
SOC	2
2.2 Bus de memoria	3
2.3 MC.	4
2.4 MD Rasguño.	4
Controlador MC 2.5.	4
Controlador MD 2.6.	5
2.7 Contadores de eventos en MC	5
2.8 Maestro IO.	5
3 Resultados y entregables	5
3.1 Pruebas	5
3.1.1 Ensayos unitarios	5
3.1.2 Pruebas de integración	5
3.2 Informe del proyecto	6
4 Política de calificación	6
5 extensiones opcionales	7

1 Resumen

En este proyecto, trabajarás con una jerarquía de memoria más compleja que en el primero. Los componentes se comunican a través de un bus semisíncrono y comprenden una memoria caché (MC), una memoria de datos (MD) y... y memoria de datos scratch (MD Scratch). La MD proporciona el mismo soporte para la instrucción lwi que en El primer proyecto, pero funciona a una velocidad menor que el MD Scratch, cuyos contenidos no se pueden almacenar en caché. (es decir, nunca se almacenan en MC). Los datos cargados mediante LWI tampoco se pueden almacenar en caché. Nuestra memoria MD admite Operaciones lwi, pero debe identificar si MD Scratch también lo admite. En este último caso La lwi debe manejarse como una lw normal. La tarea consiste en diseñar un controlador de memoria caché. que gestiona las solicitudes del procesador y las transferencias de datos a través del bus en consecuencia. El MC incluye Contadores de rendimiento adicionales. La presentación y la defensa se basarán en los procedimientos del primer proyecto. Por favor, lea atentamente esta guía, sus anexos y las instrucciones de envío que le proporcionaremos a su debido tiempo.

2 Sistema de línea base

En este segundo proyecto, el sistema de memoria comprende una memoria caché (MC), la memoria de datos (MD), una memoria virtual (MD Scratch), un bus semisíncrono y un periférico de E/S (IO Master). Tanto el – El controlador MC y el controlador maestro de E/S son administradores de bus1 . Por lo tanto, el bus requiere un árbitro para Evitar conflictos. Para garantizar un acceso justo al bus, el árbitro utiliza la asignación dinámica de prioridad. La interfaz La memoria con MIPS es idéntica a la del primer proyecto. La figura 1 muestra el diagrama de señales del bus. se describe a continuación.

2.1 SOC

Debes utilizar los módulos SoC del primer proyecto tal como están, excepto el subsistema IO MD, que cambia Completamente. Simplemente conserve los mismos módulos para el MIPS y el SoC que le proporcionamos junto con los cuatro componentes que envié, reemplace el antiguo archivo IO MD subsystem.vhd por uno nuevo y agregue todos sus componentes auxiliares. componentes.

¹Seguimos las recomendaciones del Manual de Estilo de Estándares IEEE SA 2021 sobre lenguaje inclusivo, evitando el dominio maestro/esclavo. a favor de términos descriptivos.

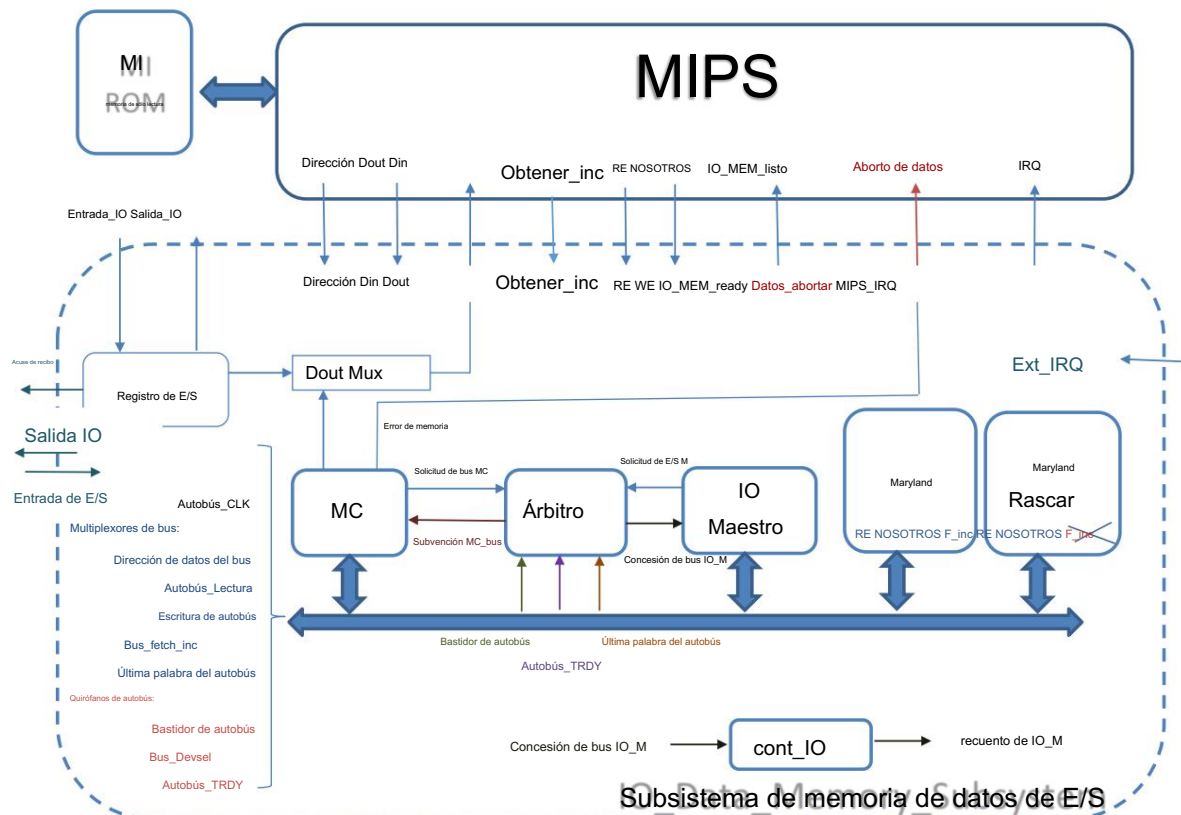


Figura 1: Componentes y señales principales del subsistema IO MD de este proyecto. El proyecto consiste en diseñar la Unidad de Control MC.

2.2 Bus de memoria

Se trata de un bus semisíncrono basado libremente en el estándar PCI, donde MC e IO Master actúan como administradores de bus, y MD y MD Scratch siempre actúan como clientes. Las líneas de datos y direcciones se multiplexan.

El bus admite ráfagas de tamaño variable. Las transferencias constan de tres fases:

- Arbitraje: Antes de iniciar una transferencia, los administradores de autobús deben solicitar el autobús mediante la señal "Bus req". El árbitro responderá afirmando la concesión del autobús, aunque puede haber una espera si el autobús está ocupado, ya que hay dos administradores.

Dirección: La transferencia comienza en el siguiente ciclo tras la confirmación de la concesión de Bus. El administrador confirma la señal de trama de Bus y envía la dirección a través de las líneas de datos de la dirección de Bus junto con el tipo de operación (Bus Rd Wr = '0' para lecturas, '1' para escrituras). El cliente que reconoce la dirección como perteneciente a su espacio de direccionamiento responderá confirmando Bus DevSel en el mismo ciclo.

Importante: Si ningún dispositivo activa devsel en el ciclo al enviar la dirección, la dirección solicitada debe almacenarse en un registro (Registro de Error ADDR) y el bit de error "Error MEM" debe estar activado. La señal "Error MEM" activará una interrupción de datos en el MIPS. El registro "Registro de Error ADDR" es legible desde el MIPS en la dirección x"01000000"; cuando el procesador lo lee, el bit de error debe borrarse.

– Hay otras dos fuentes de error de memoria (es decir, desencadenantes de error MEM): acceso no alineado (como en el proyecto anterior) y cualquier intento de escribir en el registro ADDR Error Reg, que es de solo lectura.

- Datos: Esta fase comienza justo en el ciclo posterior a la confirmación de Bus DevSel por parte del objetivo. Durante esta fase, el administrador mantiene la trama de bus, y el objetivo: a) Indica que está listo para...

realizar la operación solicitada afirmando Bus TRDY (objetivo listo) en el ciclo actual; y b)

Envía o recibe una palabra de datos por ciclo a través de la dirección de datos de bus hasta que el administrador desactive la trama de bus. En ese momento, el destino debe eliminar los datos del bus. Por lo tanto, la trama de bus no debe ser desactivada prematuramente. Si el destino no puede proporcionar los datos esperados en un ciclo dado, desactivará Bus TRDY para indicar al administrador que debe esperar. Los datos siempre se envían/reciben consecutivamente en el bus. Por ejemplo, si el administrador solicita la palabra 4, el destino enviará la palabra 4, luego la 8, luego la 12, y así sucesivamente. Cuando el administrador envía o solicita la última palabra, activará la señal de última palabra. Si solo se solicita una palabra, también se activará la última palabra, ya que es la primera y la última. –

2.3 MC

La memoria caché MC es una caché asociativa de conjuntos bidireccional (s=2) con 128 bytes (32 palabras), reemplazo FIFO, política de actualización mediante escritura directa y política convencional de asignación y recuperación ante errores de escritura. Cada vía contiene 4 bloques de 4 palabras.

2.4 MD Scratch

La memoria de datos MD Scratch es pequeña y rápida. Su latencia de acceso es tan baja como la de la MC, pero se gestiona explícitamente mediante instrucciones lw y sw dentro del rango de direcciones X"10000000"–X"100000FF". Las instrucciones lw y sw dirigidas a estas direcciones no causan fallos ni intercambios de datos con la MD. Analice el código en el archivo MD scratch.vhd y determine si el MIPS puede emitir una lwi a la memoria de datos MD Scratch. De no ser así, la lwi debe gestionarse como una lw normal. Las direcciones en el MD Scratch no se pueden almacenar en caché: el controlador MC reenvía al bus cualquier solicitud de lectura o escritura proveniente del MIPS y dirigida al espacio de direccionamiento del MD Scratch, pero no se cargará ningún bloque en el MC. Tras una lw al MD Scratch, el controlador MC reenvía la palabra devuelta por el MD Scratch al MIPS. No se admite el modo ráfaga. Es decir, se debe acceder a las palabras una a una.²

Controlador MC 2.5

El controlador MC recibe solicitudes del MIPS y, si es posible, responde con los datos almacenados en MC o en los registros de E/S. De lo contrario, realiza las transferencias necesarias con MD o MD Scratch a través del bus.

Junto con los materiales del proyecto en Moodle, encontrará un esquema a página completa de la organización interna de MC, que sirve de referencia para el diseño del controlador. El controlador es una máquina de estados finitos (FSM) de complejidad moderada. El reto reside en comprender plenamente los protocolos y sus señales.

Precaución: La lwi puede generar un problema de coherencia de caché que debe resolverse invalidando el bloque de memoria de la MC. Imagine un fallo de lw, que hace que el controlador de la MC cargue el bloque faltante en la MC. Entonces, una lwi hace referencia a la misma dirección o a cualquier otra dirección en ese bloque. El controlador MC simplemente reenvía la solicitud al MD a través del bus, ignorando la búsqueda de etiquetas. Por lo tanto, el MD incrementa y actualiza internamente la palabra solicitada por la lwi, devuelve el valor actualizado a través del bus y el controlador MC reenvía dicho valor incrementado al MIPS, sin actualizarlo en el bloque MC. Una posible lw posterior que solicite esa palabra de nuevo podría alcanzar el MC si el bloque aún no se ha desalojado, leyendo el valor obsoleto almacenado en el MC en lugar del valor actualizado en el MD. Una coincidencia de sw incluso sobrescribirá el valor en el MD previamente incrementado por la lwi. Por lo tanto, el controlador MC debe identificar las direcciones que alcanzan el MC y emitidas por una lwi, e invalidar el bloque correspondiente mediante la señal de bit de invalidación.

El controlador MC incluye una segunda FSM con dos estados (error de memoria y sin error), que también debe implementarse como parte del proceso principal de FSM, debido a su relación, o por separado. Esta FSM genera la señal de error de memoria, que el MIPS interpreta como una interrupción de datos. El controlador pasa del estado sin error al estado de error de memoria cuando: a) ningún dispositivo reconoce la dirección enviada; b) el MIPS solicita una palabra desalineada; o c) el MIPS intenta escribir en un registro de solo lectura. En este estado, el controlador continúa con su funcionamiento normal, pero mantiene activada la señal de error de memoria.

También almacena la dirección (o byte) de la palabra que desencadenó el error en el registro Addr Error Reg. El controlador permanece en estado de error hasta que el MIPS lee dicho registro, que está asignado a memoria en la dirección x"01000000". Una vez que el MIPS lee este registro, el controlador regresa al estado sin errores.

Las primeras GPGPU carecían de memoria caché, pero incluían este tipo de memoria de trabajo. En NVIDIA/CUDA se denomina memoria compartida y, en OpenCL, scratchpad. A partir de la arquitectura Fermi de NVIDIA, se introdujo un nivel que puede configurarse parcialmente como caché y parcialmente como scratchpad, a discreción del programador. Los scratchpads también son comunes en los SoC para sistemas embebidos. Nuestro MD Scratch se inspira en este concepto.

Controlador MD 2.6

El controlador MD siempre actúa como destino, reenviando las solicitudes de bus a MD si se encuentran dentro de su espacio de direcciones (X"00000000"–X"000001FF"). Las direcciones fuera de este rango se ignoran.

Temporización: Este es el mismo MD del primer proyecto, pero con retrasos adicionales: la latencia para la primera palabra (L) es mayor que para las palabras subsiguientes (R)³. Cuando el MD no puede proporcionar datos en el ciclo actual, deshabilitará la señal Bus TRDY para indicar que el controlador debe esperar. _

Modo ráfaga: cuando se activa el Bus Frame, la memoria de datos realizará la operación solicitada con la dirección inicial y luego la incrementará internamente.

2.7 Contadores de eventos en MC

El MC incluye cuatro contadores⁴:

1. cont m Errores en MC. Un error se define como un acceso (ya sea de memoria lenta o de software) a una dirección almacenable en caché ausente en MC. Los accesos a la memoria intermedia (MD Scratch) o a los registros internos de MC se ignoran como errores.
2. cont w Operaciones de escritura en MC. Se ignoran las escrituras en los registros de E/S.
3. cont r Operaciones de lectura desde MC. Las lecturas desde registros de E/S o MD Scratch se ignoran.
4. cont inv: Se incrementa cada vez que un bloque almacenado en MC se invalida debido a un lwi.

Precaución: Tenga en cuenta que cada evento solo debe incrementar una vez los contadores correspondientes. Por ejemplo, si un fallo tarda 23 ciclos en procesarse, cont m se incrementa solo una vez, no 23 veces. Registre cuántas lecturas, escrituras, fallos e invalidaciones ocurren en su banco de pruebas y verifique que los valores registrados en el banco de pruebas coincidan con los valores de los contadores MC que ve en su forma de onda. Estos contadores son muy útiles para navegar por la forma de onda durante la depuración.

2.8 Maestro de E/S

El sistema incluye un periférico de entrada/salida (IO Master) que actúa como gestor de bus. Monitorea la entrada de E/S del sistema en cada ciclo e intenta acceder continuamente al bus para escribir el valor de entrada en la primera palabra de la memoria virtual (MD Scratch). El contador cont IO se incrementa en cada ciclo en el que el IO Master accede al bus. Cuanto más eficiente sea la gestión de transferencias del controlador MC, más ciclos de bus estarán disponibles para el IO Master; es decir, mayor será el valor de cont IO.

3 Resultados y entregables

3.1 Pruebas

Primero debe verificar que su diseño funcione correctamente en todos los casos posibles de aciertos y errores de lectura y escritura utilizando los cuatro conjuntos, LWI e invalidación debido a LWI, y que los contadores se incrementen cuando corresponda. También debe realizar lecturas y escrituras en MD Scratch, generar un error de memoria, verificar que se active la señal de aborto y confirmar que se borre al leer el registro MC interno. Para ello, recomendamos la siguiente metodología de prueba.

3.1.1 Pruebas unitarias

Utilice un banco de pruebas para probar e identificar escenarios específicos y aislados. Ofrecemos un ejemplo de banco de pruebas que evalúa errores de lectura, aciertos de lectura y reemplazos limpios en todo el MC. Pruébalo en su diseño y añada gradualmente otros escenarios: errores de escritura, reemplazos sucios, lecturas y escrituras en el MD Scratch, registros de E/S, registros internos del MC y gestión de abortos. Los archivos fuente incluyen un banco de pruebas sin IRQ para simplificar la depuración.

3.1.2 Pruebas de integración

Escribe un programa en ensamblador que incluya los casos que probaste en tu banco de pruebas. Esto puede revelar efectos secundarios que podrían no ser evidentes al realizar pruebas con una serie de programas más cortos. Después, puedes comentar este código en el informe del proyecto, como hiciste en el primer proyecto.

³Recuerde el modelo de bus base explicado en la clase

⁴Los controladores reales a menudo incluyen más contadores para evaluar el rendimiento y ayudar a adaptar los programas a la jerarquía de memoria.

3.2 Informe del proyecto.

Comience con un breve resumen (aproximadamente 100 palabras) de su diseño. Evite las descripciones textuales extensas y utilice fragmentos de código/pseudocódigo y diagramas a lo largo del informe. Los siguientes elementos son obligatorios:

1. Gráfico de estado de la Unidad de Control, que explica las operaciones realizadas en cada estado y las señales activadas. Puede usar una tabla con una entrada por estado o transición para entrar en detalle y simplificar el gráfico. Evite explicaciones largas: céntrese en qué señales cambian y por qué, especialmente en las claves.
2. Una figura que muestra el desglose de la dirección para direccionar el MC como lo hacemos en clase (byte/palabra – conjunto - etiqueta).
3. Análisis de latencia de las diferentes transferencias de bus, excluyendo los retrasos inducidos por el arbitraje. Obtenga las siguientes latencias a partir de la simulación:
 - CrB(MD): Ciclos necesarios para leer un bloque de MD, donde $CrB(MD) = L \text{ ciclos (latencia de la primera palabra)} + 3 \times R \text{ ciclos (latencia de las palabras restantes)}$.
 - CwW(MD): Ciclos necesarios para escribir una palabra en MD.
 - Clwi(MD): Ciclos necesarios para realizar una lwi en MD.
 - CrW(MDscratch): Ciclos necesarios para leer una palabra desde MD Scratch.
 - CwW(MDscratch): Ciclos necesarios para escribir una palabra en MD Scratch.
 - CrW o CwW(IO REG): Ciclos necesarios para leer o escribir en registros internos de E/S.
4. Formulación matemática para calcular el promedio de ciclos por acceso a memoria en función de los costos indicados anteriormente y de los posibles eventos que ocurren en su MC: aciertos de lectura/escritura, errores de lectura/escritura en MD o en el MD Scratch, LWI o acceso al registro interno de la MC. Si tiene dudas sobre valores específicos, verifíquelos mediante simulación. Tenga en cuenta que la latencia de arbitraje no es constante en este sistema: suponga un promedio de 1,5 ciclos para los cálculos.
5. Pruebas unitarias: banco de pruebas o fragmentos de programa, debidamente comentados, que prueban eventos específicos. Por ejemplo, un fallo de lectura seguido de un flujo de aciertos, fallos de lectura en el mismo conjunto que desencadenan un reemplazo, un fallo de lectura seguido de un acierto de escritura con y sin reemplazo, referencias al MD Scratch, lwi al MD Scratch... o cualquier caso que considere importante para comprobar el correcto funcionamiento de su unidad de control. Identifique todos los estados en su máquina de estados y presente una tabla que especifique dónde y cómo se ha verificado cada caso.
6. Prueba de integración: ejemplo de código. Programa ensamblador que aprovecha la jerarquía de memoria. Calcule la aceleración proporcionada por MC y MD de Scratch en comparación con un sistema similar que solo incluye MD de este proyecto (no de la primera tarea del proyecto). Incluya en la prueba al menos una invalidación. Si es necesario, puede reutilizar uno de sus programas de prueba para destacar el impacto de MC. La aceleración puede calcularse teóricamente o a partir de los recuentos de ciclos obtenidos mediante simulaciones (con y sin caché).
7. Informe el número de horas que cada miembro del grupo dedicó a cada parte del proyecto, junto con el número total de horas.
8. Una autoevaluación, que se realizará individualmente, respondiendo a dos preguntas:
 - ¿Crees que lograste los objetivos del curso?
 - ¿Qué calificación te pondrías?

4 Política de calificación

Recuerda que evaluamos individualmente cada tarea entregada con nuestro propio programa de pruebas. Al igual que en la primera tarea del proyecto, el diseño debe funcionar correctamente para aprobar. Si detectamos errores menores, proporcionaremos nuestro banco de pruebas con el código fuente anotado paso a paso y archivos de memoria precargados. Tendrás 24 horas para corregir los errores y volver a enviar el proyecto.

El gráfico de estados debe ser claro y fácil de seguir. En lugar de incluir todas las señales en el diagrama, puede recurrir a una tabla para detallar el comportamiento de cada estado (Moore) o transición (Mealy) en el texto, identificando claramente el número de estado o transición. Sus pruebas deben especificar si cada lw, lwi o sw resulta en un acierto o un error. Indique el conjunto y la ruta para los datos almacenados en caché. Si este análisis es incorrecto o inexistente, ni siquiera verificaremos su diseño.

No explicar claramente las asignaciones de valores de señal, especialmente las de la unidad de control MC, ya sea en el informe o durante la entrevista, también puede resultar en un suspenso. Respuestas como... si pongo "0" en lugar de "1", no funciona no son válidas. Esta no es una advertencia para intimidar: el 99 % de ustedes no tendrán problemas. Solo queremos dejar claro que no se acepta enviar una solución sin demostrar comprensión. Esto también aplica a las reenvíos, que revisamos a fondo. La aparición de errores, incluso pequeños, puede requerir una inspección más profunda de su código, y suspenderá la tarea si encontramos alguna inconsistencia.

Criterios de calificación: Los principales criterios de evaluación serán si el diagrama de estados es correcto, claro y está bien explicado, y si la simulación es coherente con la descripción y la fórmula para los ciclos de acceso a memoria promedio. De forma menos crítica, también consideraremos si el controlador es eficiente y minimiza los ciclos de bloqueo. Comience con un diseño sencillo y optimícelo más adelante si el tiempo lo permite (ver opciones a continuación).

5 extensiones opcionales

- Una caché básica sin bloqueos. Incluye un búfer para escrituras en el MD. Cuando se requiere una escritura en el MD, los datos y su dirección se almacenan en registros dedicados, y el procesador recibe la señal de continuar mediante la confirmación de "Mem Ready". La MC puede continuar gestionando los aciertos mientras la Unidad de Control gestiona la escritura en el bus. La ejecución se detiene si se requiere una segunda transferencia de bus antes de completar la primera. Identifique algunos casos en los que este mecanismo mejora el rendimiento.

Hacking ético. Implementar y evaluar una función que degrada el rendimiento saturando el MC con bloques sucios, lo que provoca expulsiones en cada fallo. Evaluar su detectabilidad en nuestro SoC y posibles contramedidas.

- Reenvío de palabras críticas. En caso de fallo de lectura, el controlador MC recupera el bloque completo y envía la palabra solicitada a la CPU. El rendimiento se puede mejorar reenviando la palabra solicitada en cuanto llega, lo que permite que la CPU continúe mientras el controlador termina de recuperar el resto del bloque. Describa escenarios en los que esta técnica podría mejorar el rendimiento.